

Proiectarea algoritmilor

Sinteză completă Laboratoare 1–5 + Cheatsheet

stefan jipa

Ianuarie 2026

Cuprins

1	Introducere	2
2	Laboratorul 1 – Algoritmi & Lucrul cu fișiere	3
2.1	Noțiuni fundamentale despre algoritmi	3
2.2	Variabile și tipuri de date	3
2.3	Lucrul cu fișiere în limbajul C	3
2.3.1	Deschiderea unui fișier	3
2.3.2	Citire și scriere caracter cu caracter	3
2.3.3	Citire linie cu linie	3
2.4	Lucru cu fișiere în Java	4
2.4.1	Citire din fișier text	4
2.4.2	Scriere în fișier text	4
2.5	Prelucrarea imaginilor BMP – Conversie în grayscale	4
2.5.1	Format BMP (simplificat)	4
2.5.2	Conversia la grayscale	4
3	Laboratorul 2 – Tipuri de date, pointeri, liste	5
3.1	Tipuri definite de utilizator	5
3.1.1	Enumerări (enum)	5
3.1.2	Structuri (struct)	5
3.1.3	typedef cu struct	5
3.2	Pointeri	5
3.2.1	Pointer generic (void *)	5
3.3	Pointeri către structuri	5
3.4	Memorie dinamică în C++ (new/delete)	6
3.5	Liste simplu înlănțuite	6
3.5.1	Structură nod	6
4	Laboratorul 3 – Arbori binari & Backtracking	6
4.1	Arbori binari	6
4.1.1	Definiție	6
4.1.2	Structura nodului	6
4.1.3	Crearea recursivă a arborelui	6
4.1.4	Parcurgeri în adâncime	7
4.2	Metoda Backtracking	7
4.2.1	Idee generală	7
4.2.2	Schelet recursiv Backtracking	7
4.2.3	Problema permutărilor	8

5	Laboratorul 4 – Divide et Impera, căutare, sortare	8
5.1	Metoda Divide et Impera	8
5.1.1	Principiu	8
5.1.2	Problemă elementară	8
5.2	Determinarea maximului dintr-un vector (recursiv)	8
5.3	Căutarea binară	9
5.3.1	Condiție	9
5.3.2	Implementare recursivă simplă	9
5.4	QuickSort	9
5.4.1	Idee	9
5.4.2	Implementare simplificată	9
5.5	BubbleSort	9
5.5.1	Principiu	9
5.5.2	Implementare	10
6	Laboratorul 5 – Grafuri & algoritmi pe grafuri	10
6.1	Noțiuni de bază	10
6.2	Reprezentarea grafurilor	10
6.2.1	Matricea de adiacență	10
6.2.2	Liste de adiacență	10
6.3	Algoritmul lui Dijkstra – drum minim dintr-o sursă	10
6.3.1	Condiții	10
6.3.2	Structuri folosite	11
6.3.3	Pasul de relaxare	11
6.4	Algoritmul Floyd–Warshall – drumuri minime între toate perechile	11
6.4.1	Inițializare	11
6.4.2	Recurență	11
6.5	Algoritmul lui Kruskal – arbore de acoperire minim (MST)	11
6.5.1	Definiție MST	11
6.5.2	Pași algoritmici	12
6.6	Algoritmul lui Prim – MST pornind dintr-un nod	12
6.6.1	Idee	12
6.6.2	Pași	12
7	MEGA–Cheat–Sheet – Esențial pentru examen	12

1 Introducere

Acest document conține o sinteză structurată a laboratoarelor 1–5 de la *Proiectarea Algoritmilor*, împreună cu un **MEGA–Cheat–Sheet** final pentru învățarea rapidă pentru examen.

Sunt acoperite:

- noțiuni de bază despre algoritmi, lucrul cu fișiere (C și Java);
- tipuri de date definite de utilizator, pointeri, liste;
- arbori binari și metoda Backtracking;
- metoda Divide et Impera, căutarea binară, algoritmi de sortare;
- grafuri, reprezentări și algoritmi clasici pe grafuri (Dijkstra, Floyd, Kruskal, Prim).

2 Laboratorul 1 – Algoritmi & Lucrul cu fișiere

2.1 Noțiuni fundamentale despre algoritmi

- **Algoritm** = succesiune finită de pași pentru rezolvarea unei probleme.
- Model tip *cutie neagră*: **date de intrare** → **prelucrare** → **date de ieșire**.
- Pașii sunt operații simple executate în aceeași ordine pentru aceeași problemă.

2.2 Variabile și tipuri de date

- Variabilă = etichetă pentru o locație de memorie.
- Tipul de date determină:
 - plaja valorilor admise;
 - modul de reprezentare în memorie;
 - operațiile permise.
- Tipuri de bază: `int`, `float`, `double`, `char`.
- Tipuri derivate / compuse: `enum`, `struct` (C), `class` (C++/Java).

2.3 Lucrul cu fișiere în limbajul C

2.3.1 Deschiderea unui fișier

```
1 // deschidere pentru citire text
2 FILE *f = fopen("fisier.txt", "r");
3
4 // deschidere pentru scriere text (trunchiere)
5 FILE *g = fopen("iesire.txt", "w");
6
7 if (!f || !g) {
8     puts("Nu pot deschide fisierul");
9     exit(1);
10 }
```

2.3.2 Citire și scriere caracter cu caracter

```
1 int c;
2 while ((c = fgetc(f)) != EOF) {
3     fputc(c, g);
4 }
5 fclose(f);
6 fclose(g);
```

2.3.3 Citire linie cu linie

```
1 FILE *fisier = fopen("fisier.txt", "r");
2 char linie[100];
3
4 while (!feof(fisier)) {
5     fgets(linie, 100, fisier);
6     puts(linie);
7 }
8
9 fclose(fisier);
```

2.4 Lucru cu fișiere în Java

2.4.1 Citire din fișier text

```
1 import java.io.*;
2
3 public class CitireFisier {
4     public static void main(String[] args) throws IOException {
5         BufferedReader in = new BufferedReader(
6             new FileReader("fisier.txt"));
7         String linie;
8         while ((linie = in.readLine()) != null) {
9             System.out.println(linie);
10        }
11        in.close();
12    }
13 }
```

2.4.2 Scriere în fișier text

```
1 import java.io.*;
2
3 public class ScriereFisier {
4     public static void main(String[] args) throws IOException {
5         BufferedWriter out = new BufferedWriter(
6             new FileWriter("iesire.txt"));
7         out.write("Text de test");
8         out.newLine();
9         out.close();
10    }
11 }
```

2.5 Prelucrarea imaginilor BMP – Conversie în grayscale

2.5.1 Format BMP (simplificat)

- Header de 54 octeți (informații despre imagine).
- Date imagine: pixeli în format RGB (24 biți/pixel: 8 biți pentru fiecare din R, G, B).

2.5.2 Conversia la grayscale

Se folosește formula de luminanță:

$$Y = 0.114 \cdot B + 0.587 \cdot G + 0.299 \cdot R$$

Algoritm:

1. Se citește header-ul și se scrie în fișierul de ieșire.
2. Pentru fiecare pixel (R,G,B) se calculează Y .
3. Se scrie în fișierul rezultat (Y, Y, Y).

Temă

Generarea a trei imagini BMP conținând separat componentele R, G și B (celelalte două setate la 0).

3 Laboratorul 2 – Tipuri de date, pointeri, liste

3.1 Tipuri definite de utilizator

3.1.1 Enumerări (enum)

```
1 enum Zi { LUNI = 1, MARTI, MIERCURI, JOI, VINERI };  
2 Zi z = LUNI;
```

Enumerările definesc mulțimi de constante simbolice.

3.1.2 Structuri (struct)

```
1 struct Student {  
2     char nume[30];  
3     float medie;  
4 };  
5  
6 struct Student s1 = {"Popescu Andrei", 9.75f};
```

3.1.3 typedef cu struct

```
1 typedef struct {  
2     int x, y;  
3 } Punct;  
4  
5 Punct p1;
```

3.2 Pointeri

- Pointer = variabilă ce memorează adresa altei variabile sau a unei funcții.
- Operator & – adresa variabilei.
- Operator * – conținutul adresei (dereferențiere).

```
1 int x = 10;  
2 int *p = &x;    // p pointeaza catre x  
3 printf("%d", *p); // afiseaza 10
```

3.2.1 Pointer generic (void *)

```
1 void *pv;  
2 float f = 3.14f;  
3 pv = &f;  
4 printf("%f\n", *( (float*) pv ));
```

3.3 Pointeri către structuri

```
1 struct Coord { int x, y; };  
2  
3 Coord c, *pc;  
4 pc = &c;  
5  
6 pc->x = 10;  
7 pc->y = 20;
```

3.4 Memorie dinamică în C++ (new/delete)

```
1 int *p = new int;      // alocare
2 *p = 5;
3 delete p;              // eliberare
```

3.5 Liste simplu înlănțuite

3.5.1 Structură nod

```
1 struct Nod {
2     int data;
3     Nod *urm;
4 };
```

Operații de bază:

- parcurgere (afișare);
- inserare la început / la poziție dată;
- ștergere nod;
- căutare element.

Temă

Implementarea funcției de căutare a elementului aflat la o poziție dată în listă.

4 Laboratorul 3 – Arbori binari & Backtracking

4.1 Arbori binari

4.1.1 Definiție

Un arbore este **binar** dacă fiecare nod are cel mult doi descendenți direcți (fii): subarbore stâng și subarbore drept.

4.1.2 Structura nodului

```
1 struct ArboreBinar {
2     int inf;
3     ArboreBinar *st;
4     ArboreBinar *dr;
5 };
```

4.1.3 Crearea recursivă a arborelui

```
1 ArboreBinar* createTree() {
2     int x;
3     cout << "Introduceti valoarea nodului (-1 pentru NULL): ";
4     cin >> x;
5     if (x == -1) return NULL;
6
7     ArboreBinar *r = new ArboreBinar;
8     r->inf = x;
9     cout << "Subarbore stang al nodului " << x << ":\n";
10    r->st = createTree();
11    cout << "Subarbore drept al nodului " << x << ":\n";
12    r->dr = createTree();
```

```

13     return r;
14 }

```

4.1.4 Parcurgeri în adâncime

Inordine (S–R–D):

```

1 void inordine(ArboreBinar *r) {
2     if (r != NULL) {
3         inordine(r->st);
4         cout << r->inf << " ";
5         inordine(r->dr);
6     }
7 }

```

Preordine (R–S–D):

```

1 void preordine(ArboreBinar *r) {
2     if (r != NULL) {
3         cout << r->inf << " ";
4         preordine(r->st);
5         preordine(r->dr);
6     }
7 }

```

Postordine (S–D–R):

```

1 void postordine(ArboreBinar *r) {
2     if (r != NULL) {
3         postordine(r->st);
4         postordine(r->dr);
5         cout << r->inf << " ";
6     }
7 }

```

4.2 Metoda Backtracking

4.2.1 Idee generală

- Se construiește soluția pas cu pas, poziție cu poziție.
- La fiecare pas se testează dacă soluția parțială este *validă*.
- Dacă nu, se revine (backtrack) și se încearcă altă valoare.

4.2.2 Schelet recursiv Backtracking

```

1 int sol[100], n;
2
3 int Valid(int k) {
4     // conditii interne (distinctie, restrictii, etc.)
5     return 1;
6 }
7
8 int Solutie(int k) {
9     return (k == n);
10 }
11
12 void Afisare() {
13     for (int i = 1; i <= n; ++i)
14         cout << sol[i] << " ";

```

```

15     cout << "\n";
16 }
17
18 void Back(int k) {
19     for (int i = 1; i <= n; ++i) {
20         sol[k] = i;
21         if (Valid(k)) {
22             if (Solutie(k)) Afisare();
23             else Back(k + 1);
24         }
25     }
26 }

```

4.2.3 Problema permutărilor

- Mulțimea: $\{1, 2, \dots, n\}$.
- **Valid(k)**: elementele din `sol[1..k]` sunt distincte.
- **Solutie(k)**: `k == n`.

Teme Backtracking

1. Generarea tuturor numerelor ale căror cifre (fără 0) au suma egală cu o valoare S dată.
2. Generarea tuturor numerelor prime de lungime x cifre, cu cifre distincte.

5 Laboratorul 4 – Divide et Impera, căutare, sortare

5.1 Metoda Divide et Impera

5.1.1 Principiu

1. **Divide**: problema se descompune în 2 (sau mai multe) subprobleme de același tip, de dimensiuni mai mici;
2. **Conquer**: fiecare subproblemă se rezolvă independent;
3. **Combine**: se combină rezultatele subproblemelor pentru soluția finală.

5.1.2 Problemă elementară

Subproblemă care nu mai poate fi împărțită și se rezolvă direct (de exemplu, 1 element într-un vector).

5.2 Determinarea maximului dintr-un vector (recursiv)

```

1 int v[100];
2
3 int maxim(int li, int ls) {
4     if (li == ls) return v[li];
5
6     int mij = (li + ls) / 2;
7     int a = maxim(li, mij);
8     int b = maxim(mij + 1, ls);
9
10    return (a > b ? a : b);
11 }

```


5.3 Căutarea binară

5.3.1 Condiție

Vectorul trebuie să fie ordonat.

5.3.2 Implementare recursivă simplă

```
1 int cautareBinara(int v[], int li, int ls, int x) {
2     if (li > ls) return -1;
3     int mij = (li + ls) / 2;
4
5     if (x == v[mij]) return mij;
6     if (x < v[mij]) return cautareBinara(v, li, mij - 1, x);
7     return cautareBinara(v, mij + 1, ls, x);
8 }
```

5.4 QuickSort

5.4.1 Idee

1. Se alege un pivot.
2. Se particționează vectorul: elementele mai mici la stânga, cele mai mari la dreapta.
3. Se sortează recursiv cele două subsecvențe.

5.4.2 Implementare simplificată

```
1 int poz(int v[], int p, int u) {
2     int pivot = v[p];
3     int i = p, j = u;
4
5     while (i < j) {
6         while (i < j && v[i] <= pivot) i++;
7         while (v[j] > pivot) j--;
8         if (i < j) swap(v[i], v[j]);
9     }
10    swap(v[p], v[j]);
11    return j;
12 }
13
14 void quicksort(int v[], int p, int u) {
15     if (p < u) {
16         int k = poz(v, p, u);
17         quicksort(v, p, k - 1);
18         quicksort(v, k + 1, u);
19     }
20 }
```

5.5 BubbleSort

5.5.1 Principiu

- se parcurge vectorul de mai multe ori;
- se compară perechi de elemente vecine;
- se interschimbă dacă nu sunt în ordine;
- se repetă până nu se mai fac interschimbări.

5.5.2 Implementare

```
1 void bubblesort(int v[], int n) {
2     int sortat = 0;
3     while (!sortat) {
4         sortat = 1;
5         for (int i = 0; i < n - 1; ++i) {
6             if (v[i] > v[i + 1]) {
7                 swap(v[i], v[i + 1]);
8                 sortat = 0;
9             }
10        }
11    }
12 }
```

6 Laboratorul 5 – Grafuri & algoritmi pe grafuri

6.1 Noțiuni de bază

- **Graf neorientat** $G = (N, R)$: mulțime de noduri N și muchii R .
- **Graf orientat (digraf)** $G = (V, E)$: mulțime de vârfuri V și arce E .
- **Cale**: succesiune de vârfuri astfel încât între oricare două consecutive există muchie/arc.
- **Cale simplă**: nodurile intermediare sunt distincte.

6.2 Reprezentarea grafurilor

6.2.1 Matricea de adiacență

Pentru graf neorientat:

$$A_{i,j} = \begin{cases} 1, & \text{dacă există muchia } [i, j] \\ 0, & \text{altfel} \end{cases}$$

Pentru graf orientat:

$$A_{i,j} = \begin{cases} 1, & \text{dacă există arcul } (i, j) \\ 0, & \text{altfel} \end{cases}$$

6.2.2 Liste de adiacență

- Graf neorientat: pentru fiecare nod x se memorează lista vecinilor adiacenți.
- Graf orientat:
 - $L^+(x)$ – lista succesorilor (arce care ies din x);
 - $L^-(x)$ – lista predecesorilor (arce care intră în x).

6.3 Algoritmul lui Dijkstra – drum minim dintr-o sursă

6.3.1 Condiții

- graf orientat/neorientat ponderat;
- costurile arcelor trebuie să fie nenegative.

6.3.2 Structuri folosite

- `dist[i]` – distanța minimă cunoscută de la sursă la i ;
- `visited[i]` – marcarea dacă i a fost procesat;
- `parent[i]` – părintele lui i în drumul minim;
- `cost[i][j]` – costul arcului (i, j) (INF dacă nu există).

6.3.3 Pasul de relaxare

```
1 // u - nodul curent selectat cu distanța minimă
2 for (int v = 0; v < n; ++v) {
3     if (!visited[v] && cost[u][v] != INF &&
4         dist[u] + cost[u][v] < dist[v]) {
5         dist[v] = dist[u] + cost[u][v];
6         parent[v] = u;
7     }
8 }
```

6.4 Algoritmul Floyd–Warshall – drumuri minime între toate perechile

6.4.1 Inițializare

$$D[i][j] = \begin{cases} \text{Cost}[i][j], & \text{dacă există arc} \\ 0, & i = j \\ \text{INF}, & \text{altfel} \end{cases}$$

6.4.2 Recurență

```
1 for (int k = 0; k < n; ++k)
2     for (int i = 0; i < n; ++i)
3         for (int j = 0; j < n; ++j)
4             if (D[i][k] + D[k][j] < D[i][j])
5                 D[i][j] = D[i][k] + D[k][j];
```

La final, `D[i][j]` conține costul drumului minim de la i la j .

6.5 Algoritmul lui Kruskal – arbore de acoperire minim (MST)

6.5.1 Definiție MST

Arbore de acoperire minim:

- subgraf neorientat, conectat;
- conține toate nodurile grafului;
- fără cicluri;
- suma costurilor muchiilor este minimă;
- pentru N noduri, are $N - 1$ muchii.

6.5.2 Pași algoritmici

1. MST = mulțime vidă de muchii.
2. Se sortează toate muchiile crescător după cost.
3. Pentru fiecare muchie (u, v) în ordine:
 - dacă u și v sunt în componente diferite (nu se formează ciclu) $\rightarrow$ se adaugă muchia în MST;
 - altfel se ignoră.
4. Se oprește când MST conține $N - 1$ muchii.

6.6 Algoritmul lui Prim – MST pornind dintr-un nod

6.6.1 Idee

- se pornește dintr-un nod oarecare (rădăcina);
- se extinde treptat arborele, alegând mereu muchia de **cost minim** care leagă un nod din arbore cu un nod din afara lui;
- rezultatul final este arborele de acoperire minim.

6.6.2 Pași

1. Se alege nodul inițial; $U = \{\text{nod start}\}$, $A = \{\}$.
2. Cât timp U nu conține toate nodurile:
 - se caută muchia (u, v) de cost minim cu $u \in U$ și $v \notin U$;
 - se adaugă muchia în A și nodul v în U .

7 MEGA–Cheat–Sheet – Esențial pentru examen

Laboratorul 1

- Algoritm = pași finiți, input $\rightarrow$ output.
- Fișiere C: `fopen`, `fgetc`, `fputc`, `fgets`, `fclose`.
- Fișiere Java: `BufferedReader`, `FileReader`, `BufferedWriter`, `FileWriter`.
- BMP $\rightarrow$ grayscale:

$$Y = 0.114 \cdot B + 0.587 \cdot G + 0.299 \cdot R.$$

Laboratorul 2

- `enum` – constante simbolice.
- `struct` – grupare de variabile diferite într-o înregistrare.
- Pointeri: `int *p = &x;` ; dereferențiere: `*p`.
- Liste simplu înlănțuite: nod cu `data` și `urm`.

Laboratorul 3

- Arbori binari: maxim doi fii (stâng, drept).
- Parcurgeri DFS:
 - inordine: S–R–D;
 - preordine: R–S–D;
 - postordine: S–D–R.
- Backtracking:
 - se construiește soluție vectorială;
 - funcții `Valid`, `Solutie`, `Afisare`.

Laboratorul 4

- Divide et Impera: **Divide** → **Conquer** → **Combine**.
- Maxim recursiv: împarte vectorul în două jumătăți, ia maximum din fiecare.
- Căutare binară: vector ordonat, comparare cu elementul de la mijloc.
- QuickSort: sortare rapidă bazată pe pivot și partiționare.
- BubbleSort: comparare și interschimbare elemente vecine, repetat până la sortare.

Laboratorul 5

- Matrice de adiacență: acces rapid la prezența unei muchii.
- Liste de adiacență: utilizare eficientă a memoriei.
- Dijkstra: drum minim dintr-o sursă, costuri pozitive, structură `dist`, `visited`, `parent`.
- Floyd–Warshall: drum minim între toate perechile, recurența:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j]).$$

- Kruskal: MST prin sortarea muchiilor și evitarea ciclurilor.
- Prim: MST pornind dintr-un nod, adăugând muchia minimă către un nod nou.

Formule și idei de ținut minte

- Luminanță grayscale: $Y = 0.114B + 0.587G + 0.299R$.
- Mijloc vector: $\text{mij} = (\text{li} + \text{ls}) / 2$.
- Floyd–Warshall: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$.
- MST: pentru N noduri, arborele are $N - 1$ muchii.